IN1160 – Oblig 1a: Vektorrom

Våren 2026

Det er en god idé å lese gjennom hele oppgavesettet før dere setter i gang. Dersom dere har spørsmål så kan dere:

- spørre i gruppetimene,
- spørre på Discourse
- eller sende epost til in1160-support@ifi.uio.no dersom alternativene over av en eller annen grunn ikke passer for spørsmålet.

Innlevering

Oppgaven leveres innen 11.02. klokken 23:59 i Devilry.

Innleveringen skal bestå av én Jupyter notebook med både kode og tilhørende forklaringer. **Før innlevering skal du kjøre gjennom hele notebooken, før du lagrer siste gang. Den bør kjøre uten å feile og vise den grafikken og de utskriftene som skal være med.** Skulle dere av tekniske grunner ikke kunne besvare alle spørsmålene i en notebook kan dere i denne obligen også levere vanlige Python-filer, sammen med en PDF som inneholder grafikker og resultater fra kjøring, samt deres tekst-besvarelser. Ta kontakt med emnestaben, så vi får ordnet eventuelle problemer før oblig 1b.

Vi understreker at innlevering av kode alene ikke er nok for å bestå oppgaven – vi forventer at notebooken også skal inneholde kommentarer (på norsk eller engelsk) på hva dere har gjort og begrunnelser for valgene dere har tatt underveis. La enhver oblig bli en trening i å formidle forskning. Bruk helst hele setninger, og matematiske formler om nødvendig. Resultater skal presenteres i tabeller på en oversiktlig måte. Det å forklare med egne ord, bruke begreper vi har gått gjennom på forelesningene og å forklare og reflektere over løsningene deres er en viktig del av læringsprosessen – ta det på alvor!

Når det gjelder bruk av generative prateroboter (ChatGPT og lignende): Dere kan bruke dem som en "sparringspartner", for eksempel for å forklare noe dere ikke helt har forstått. Dere har imidlertid ikke lov til å bruke dem til å generere løsninger (enten delvis eller fullstendig) til noen av oppgavene. Funksjoner for automatisk skriving av kode, som Copilot i VS Code, må derfor også være deaktivert mens dere jobber på obligen.
Bruker dere KI-verktøy vil vi også at dere kort beskriver hvordan dere har brukt dem under arbeidet med oppgaven.

Det er ikke mulighet for omlevering av obliger som ikke bestås.

Oppsett

Denne første obligen skal blant annet hjelpe dere med å komme i gang med verktøyene vi kommer til å bruke gjennom semesteret. Sørg for å aktivere IN1160-miljøet, så bør dere kunne kjøre kodeblokken under. Får dere noen feilmeldinger, oppfordres dere til å møte opp i lab- og gruppetimene for teknisk hjelp.

```python
from helpers_1a import sanity_check

sanity_check()
```

```
=== Sjekker pakker ===
[OK] numpy (2.3.2)
[OK] matplotlib (3.10.3)
[OK] pandas (2.3.1)
[OK] ipykernel (6.30.0)
[OK] scikit-learn (1.7.1)
[OK] seaborn (0.13.2)
[OK] openml (0.15.1)
[OK] pyyaml (6.0.2)
[OK] ipython (9.4.0)
[OK] plotly (6.3.1)
[OK] gymnasium (1.2.1)
```

```
/Users/bruker/Desktop/Projects/ML-AI/mlai_env/lib/python3.13/site-packages/tqdm/auto.py:21: TqdmWarning: IProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
```

```
[OK] datasets (4.4.1)
[OK] gymnasium[toy-text]
====================
Alt fungerer som det skal!
```

Bakgrunn

I denne innleveringen skal vi jobbe med datasettet *Norsk aviskorpus* (NAK) fra Nasjonalbiblioteket. Dette korpuset består av tekster hentet fra en rekke norske aviser fra perioden 1998-2019. Det fulle datasettet kan finnes her.

NAK består av over 1 500 000 ord (~2,6 GB), men vi skal jobbe med en forenklet versjon av datasettet, gitt i fila `NAK_oblig_1a.txt`.

I denne obligen skal dere jobbe med å lage enkle vektorrepresentasjoner av ord. Dere skal gjøre dette basert på hvilke andre ord de forekommer sammen med i setninger i korpuset. Vi skal så se på hvordan vi kan bruke disse vektorene til å finne likheter mellom ord ved bruk av cosinuslikhet.

I oppgavene som følger skal dere:

1. gjøre preprosessering av tekstene
2. representere ord som vektorer i en vektorrom-modell
3. undersøke likhet mellom ord gjennom vektorene

Oppgave 1 – Forberede data

Oppgave 1.1 – Laste inn datasett (1 poeng)

I denne oppgaven skal dere laste inn NAK-datasettet fra tekstfila `NAK_oblig_1a.txt`. Dette skal gjøres med funksjonen `load_dataset()`. Denne tar filnavnet til datasettet (som skal ligge i samme mappe som denne notebook-fila) som argument og skal returnere en liste med setningstrenger.

Skriv ferdig funksjonen `load_dataset()` i cellen under og last inn datasettet:

Kommentarer og begrunnelser Oppgave 1.1

Jeg laster inn NAK-datasettet fra en tekstfil som blir gitt av å åpne .zip mappen. Filen leses linje for line og linjene blir lagret som setninger i en liste. Bruker stip() for å fjerne linjeskift og eventuelle mellomrom før og etter, fordi tokeniseringen kan bli påvirket av dette, dette er best practice. load_dataset() funksjonen resulterer i en liste av setningsstrenger, der hvert element representerer en linje i datasettet, som gjør det svært enkelt å iterere over datasettet.

```
# Oppgave 1.1
def load_dataset(dataset_path):
    """
    Argumenter:
    - dataset_path : En streng som spesifiserer hvor vi finner
datasettet.

    Returnerer:
    - dataset : En liste med setninger.
    """
    dataset = []
    with open(dataset_path, "r") as file:
        for line in file:
            dataset.append(line.strip())

    return dataset

# Bruk load_dataset() til å laste inn datasettet her:
loaded_dataset = load_dataset("NAK_oblig_1a.txt")
```

Oppgave 1.2 – Tokenisering og inspeksjon av dataene (2 poeng)

Vi har nå en liste med setninger, men vi ønsker å utforske de individuelle ordene, så vi må dele opp teksten ytterligere. Denne typen oppdeling av tekst kaller vi for *tokenisering*, fordi hver tekstbit vi mater inn i en modell kalles et *token*. Vi er ofte interessert i å vite hvor mange unike tokens vi har i datasettet vårt. Hvert unike token kaller vi for en *ordtype*, eller bare *type*. Alle ordtypene vi har til sammen kaller vi for *vokabularet* vårt.

I følgende setning er det altså 7 tokens og 5 typer (vi teller "En" og "en" som to forskjellige typer):

En for alle og alle for en

I cellen under skal dere lage funksjonen `tokenize()` som tokeniserer en setning ved å dele den opp med metoden `.split()`.

Videre skal dere:

- Tokenisere hver setning i datasettet med `tokenize()`.
  - Dette skal altså resultere i en liste av lister med individuelle ord.
- Rapportere hvor mange ord dere får.
- Rapportere hvor mange ordtyper dere får.

Kommentarer og begrunnelser Oppgave 1.2

I denne oppgaven tokeniseres setningene ved bruk av metoden .split(), som deler opp setningens ord på whitespaces. Datasettet resulterer nå i en liste av lister, der hver indre liste består av tokens/ord. Deretter beregner jeg antall tokens og antall ordtyper. Tokens -> lengen av hver setning summert sammen. Antall ordtyper -> lengden av tokens i et set(), der kun unike elementer forekommer. "En" og "en" behandles som unike tokens.

```
# Oppgave 1.2
def tokenize(text):
    """Tar inn en streng med tekst og returnerer en liste med ord."""
    #tokens: list[str] = text.split()
    return text.split()

# Tokeniser datasettet her:
for i in range(len(loaded_dataset)):
    loaded_dataset[i] = tokenize(loaded_dataset[i])

# Beregn antall ord og ordtyper her:
word_count = 0
uniques = set()

for i in range(len(loaded_dataset)):
    word_count += len(loaded_dataset[i])
    uniques.update(loaded_dataset[i])

type_count = len(uniques)

print(f"Word/Token count = {word_count}\n")
print(f"Type count = {type_count}\n")
```

```
Word/Token count = 28671744

Type count = 657933
```

Oppgave 1.3 – Fjerning av stoppord (2 poeng)

Hvis alt er gjort riktig så langt bør dere ha fått ~31 000 000 tokens og ~1 000 000 ordtyper. Det betyr altså at hver type i gjennomsnitt forekommer omtrent 31 ganger i datasettet. Vi vet derimot (fra Zipf's lov) at noen relativt få ord forekommer veldig mange ganger, mens veldig mange andre ord forekommer veldig sjeldent.

Blant de mest vanlige ordene finner vi ord som for eksempel "og", "i" og "det". Siden disse ordene forkommer såpass ofte, bidrar de sjeldent med informasjon som hjelper oss med å skille mellom ulike ord. Vi kan derfor hoppe over disse ordene for å redusere kjøretiden på treningen av modellene våre.
Denne prosessen kaller vi å fjerne *stoppord*. En liste med norske stoppord (inkludert tegnsetting) er gitt i funksjonen `get_norwegian_stopwords()` i `helpers_1a.py`.

I denne oppgaven skal dere:

- Hente lista med stoppord med `get_norwegian_stopwords()`.
- Fjerne alle stoppordene fra datasettet.
  - En setning som *"en katt hadde spist en mus"* skal altså reduseres til *"katt spist mus"*.
- Rapportere hvor mange ordtyper og hvor mange tokens det er igjen i datasettet.
  - Skriv en kort kommentar om hvordan forholdet mellom disse tallene endrer seg.

Kommentarer og begrunnelser Oppgave 1.3

Fjerner stoppord fra datasettet ved å bruke gisten gitt av get_norwegian_stopwords. Hensikten med dette er å redusere støyen som forekommer og i tillegg gjøre programmet mer effektivt ved å ikke behandle hyppige ord som "og", "i", og lignende. Antall tokens synker betraktelig etter filtreringen av stopp-ordene, antall ordtyper synker også, men ikke like mye. Dette resulterer i at frekvensen mellom hver type blir høyere siden vi fjerner mange av de hyppige ordene.

```
from helpers_1a import get_norwegian_stopwords

# Oppgave 1.3
stoppord = get_norwegian_stopwords()

for i in range(len(stoppord)):
    if stoppord[i] in loaded_dataset[i]:
        loaded_dataset[i].remove(stoppord[i])
        uniques.discard(stoppord[i])


print(f"Stoppord fjernet, Tokens: {len(loaded_dataset)}")
print(f"Stoppord fjernet, Types: {len(uniques)}")
```

```
Stoppord fjernet, Tokens: 1388746
Stoppord fjernet, Types: 657930
```

Oppgave 1.3 – Tekstbesvarelse

Hvor mange ordtyper og tokens står dere igjen med etter fjerning av stoppordene?

Svar: Etter fjerning av stoppord står jeg igjen med 28 671 744 tokens og 657 933 ordtyper. Sammenlignet med datasettet før stoppordfjerning er antall tokens redusert betydelig, mens antall ordtyper reduseres mindre proposjonalt. Dette skyldes at stoppord som "og", "i", og "det" forekommer svært ofte, men representerer relativt få unike typer. Når disse fjernes, forsvinner mange gjentatte forekomster, noe som senker token-antallet. Forholdet mellom tokens og typer endres dermed slik at hver type i gjennomsnitt forekommer sjeldnere. Dette er ønskelig i vektorrom-modeller, siden de gjenværende ordene typisk bører mer semantisk informasjon og gir et mer informativt representasjonsrom.

Oppgave 1.4 – Fjerning av sjeldne ord (2 poeng)

Som nevnt er det også vanlig at vi har svært mange ord som forekommer svært sjeldent. Blant disse ordene finner vi ofte feilstavinger og ord som ikke har blitt lest riktig fra teksten. Disse ordene er som oftest heller ikke særlig nyttige for modellene våre og kan derfor også fjernes.

I denne oppgaven skal dere:

- Lage en liste med de 10 000 ordene som forekommer oftest i datasettet (der stoppord er fjernet).
  - Her kan det være nyttig å benytte seg av klassen `Counter` og metoden `.most_common()`.
- Rapportere hvor mange typer og tokens det er i datasettet etter dette.
  - Hvordan er forholdet mellom disse nå?

Tips: Under utvikling kan det være lurt å begrense vokabularet enda litt mer. Dere kan f.eks. hente de 100 vanligste ordene.

Kommentarer og begrunnelser Oppgave 1.4

I denne oppgaven lages det en frekvensteller over tokens-ene i datasettet ved å samle dem i en ny tom liste og bruke Counter funksjonen. Deretter finner jeg de 10 000 vanligste ordene med funksjonen most_common(100000) og datasettet blir dermed filtrert slik at hvert ord ikke funnet i denne listen blir fjernet, siden vi ikke er like interessert i dem. Grunnen til at interessen synker for disse ordene handler om at de har en høyere sannsynlighet for at inneholder feilstavinger og andre feil/defekter. Samtidig så vil de være "outliers" som ikke gir en like sterk statistisk begrunnelse for at skal bli inkludert i telelrbaserte modeller. Fordelene er rett og slett at vi reduserer kjøretiden og minnebruken betydelig ved å redusere til 10 000 og vi antar at vi får et godt nok representativt vokabulær.

```
from collections import Counter

# Oppgave 1.4
all_tokens = []
for sentence in loaded_dataset:
    all_tokens.extend(sentence)
```

```
counter = Counter(all_tokens)

dataset_10k = counter.most_common(10000)
top_10k_words = {word for word, freq in dataset_10k}

filter_dataset = []
for sentence in loaded_dataset:
    filtered_sentence = [word for word in sentence if word in
top_10k_words]
    filter_dataset.append(filtered_sentence)

word_count = 0
uniques = set()

for sentence in filter_dataset:
    word_count += len(sentence)
    uniques.update(sentence)

print(f"Word/Token count = {word_count}\n")
print(f"Type count = {len(uniques)}\n")
```

```
Word/Token count = 24368510

Type count = 10000
```

Oppgave 1.4 – Tekstbesvarelse

Hvor mange typer og tokens står dere igjen med etter fjerning av de sjeldne ordene?

Svar: Etter fjerning av sjeldne ord ved å begrense vokabulæret til de 10 000 mest frekvente ordene i datasettet, er det 10 000 resterende ordtyper. Antall tokens reduseres betydelig sammenlignet med datasettet før filtrering, siden alle sjelde ord og feilstavinger fjernes fullstendig. Forholdet mellom tokens og typer endres dermed slik at hver type i gjennomsnitt forekommer langt oftere enn tidligere. Dette gir et mer kompakt og informativt datasett, der ord som faktisk bidrar til semantisk struktur dominerer. En slik reduksjon er gunstig både for kjøretid og for kvaliteten på vektorrepresentasjonene som senere skal trenes.

Oppgave 2 – Vektorisering

Oppgave 2.1 – Fra ord til tall (2 poeng)

Vi har nå fjernet både de vanligste og sjeldneste de ordene fra vokabularet vårt. De resterende ordene burde være en passe mengde for modellen vår. Modellen vi skal bruke kan derimot ikke bruke strenger som treningsdata, så vi må lage tallrepresentasjoner av dem, der hver type i vokabularet vårt knyttes til et bestemt tall.

For å holde styr på sammenhengen mellom ordene og tall kan vi bruke en ordbok der vi tilordner et tall til hvert ord. For å knytte tallene tilbake til ordene kan vi lage en tilsvarende ordbok med tallet som nøkkel og ordet som verdi.

I denne oppgaven skal dere fylle ut de to ordbøkene `word_to_int` og `int_to_word`. Her kan funksjonen `enumerate()` være nyttig.

Kommentarer og begrunnelser Oppgave 2.1

For å representere ord i en matrise så må jeg først mappe hver ordtype til et heltall. På grunn av dette, så lager jeg en sortert vokabulærliste og bruker funksjonen enumerate() til å lage to ordbøker, nemlig: word_to_int, ord->idx og int_to_word, idx->ord. Ved å mappe dem begge veier slik som nevnt, så kan man effektivisere og muliggjøre rad og kolonneoppslag for ønsket resultat.

```
# Oppgave 2.1
vocab = sorted(uniques)
word_to_int = {}
int_to_word = {}

for idx, word in enumerate(vocab):
    word_to_int[word] = idx
    int_to_word[idx] = word
```

Oppgave 2.2 – Bygging av vektorer (5 poeng)

I denne oppgaven skal vi lage vektorrepresentasjoner av ordene vi kom fram til i **1.4**. Vi skal benytte oss av enkle tellevektorer, der vi bruker *konteksten* et ord befinner seg i til å informere oss om hva slags ord det er snakk om. Intuisjonen for dette er at ord som brukes i samme kontekst typisk har like eller relaterte betydninger:

> "*You shall know a word by the company it keeps*"
> – J. R. Firth (1957)

I cellen under finner dere en påbegynt `CountVectorizer`-klasse. Når det blir opprettet en instans av denne, lager metoden `zeros()` en matrise av nuller som er like lang og like bred som vokabularet vi sender inn. Her representerer hver rad et ord i vokabularet vårt, der f.eks. rad 5 representerer ordet som ble tilordnet tallet 5 i forrige oppgave. Kolonnene fungerer på samme måte.

Etter å ha opprettet en `CountVectorizer`, skal man kunne kalle på metoden `.fit()`, der man sender inn de tokeniserte tekstdataene fra oppgave **1.2**. Denne sender én og én setning inn til metoden `_add_sentence()`, som dere skal fylle ut.
Denne metoden skal oppdatere matrisen ved å telle antall ganger to ord forekommer i samme setning. For hvert ord i i setningen, skal vi altså legge til 1 på posisjon (i, j) i matrisen for hvert ord j som også forekommer i setningen.

Når treningen er overstått kan vi hente ut en vektor for et ord ved å bruke metoden `get_vector()`.

I denne oppgaven skal dere:

- Fylle ut metoden `_add_sentence()` slik at matrisen fylles ut på riktig måte.
- Opprette en instans av `CountVectorizer`.

- Trene denne på de tokeniserte dataene.
  - Dette kan ta litt tid avhengig av maskinen dere kjører på.
- Hente ut en vektor for et ord dere selv velger. Finn ut av hvilket annet ord dette forekommer oftest med.

Kommentarer og begrunnelser Del 1/3 Oppgave 2.2

Implementert metoden _add_sentence(), som fyller samforekomstmatrisen basert på setningene i datasettet. Hvert ord blir først mappet til en indeks ved hjelp av word_to_int listen. Teller deretter hvor ofte ordpar forekommer i setningen og bruker NumPy indekser i stedet for en dobbel løkke, for å gjøre det nært C-kode og dermed veldig effektivt. Jeg trekker i fra diagonalen slik at ord ikke telles med seg selv.

```
from numpy import zeros
import numpy as np

class CountVectorizer:
    def __init__(self, vocab, word_to_int):
        self.vocab = vocab
        self.word_to_int = word_to_int
        self.matrix = self._create_matrix(vocab)

    def _create_matrix(self, vocab):
        """
        Lager en matrise bestående av 0-vektorer. Matrisen er
        like lang og like bred som vokabularet vårt, slik at
        hver rad representerer et ord, og hver kolonne
        representerer antall ganger et annet ord forekommer
        med rad-ordet.
        """
        matrix = zeros((len(vocab), len(vocab)), dtype=np.int32)

        return matrix

    def _add_sentence(self, sentence_tokens):
        """
        Oppdaterer matrisen med observasjoner fra setningen.
        """
        # Bruker np.array i stedet for dobbel løkke, int32 ->
Effektivt
        idxs = np.array([self.word_to_int[word] for word in
sentence_tokens], dtype=np.int32)
        # Legger til 1 for alle par i setningen
        self.matrix[np.ix_(idxs, idxs)] += 1
        # Fjerner like (co occurrence)
        self.matrix[idxs, idxs] -= 1

    def fit(self, dataset):
        """Trener modellen på `dataset`."""
        for sent in dataset:
```

```
            self._add_sentence(sent)

    def get_vector(self, word):
        """Henter vektoren for et gitt ord."""
        word_int = self.word_to_int[word]

        return self.matrix[word_int]
```

Kommentarer og begrunnelser Del 2/3 Oppgave 2.2

Etter vektoriseringen, trenes modellen ved bruk av funksjonen fit() på datasettet fra oppgave 1.4. Setningene itereres og sendes til _add_sentence() funksjonen. Treningen er dermed å danne samforekomstmatrisen. Hver rad i matrisen resulterer i en tellevektor som beskriver konteksten ordet opptrer i.

```
# Oppgave 2.2
# Lag en instans av CountVectorizer med våre parametre
vectorizer = CountVectorizer(vocab, word_to_int)
vectorizer.fit(filter_dataset)
# Tren vektorisereren på våre data
```

Kommentarer og begrunnelser Del 3/3 Oppgave 2.2

Når modellen er ferdig trent, så hentes vektoren til et ord av get_vector(). argmax() blir brukt for å finne det største vektoret. Slår deretter opp i int_to_word for å finne ordet som forekommer flest ganger sammen med målordet.

```
# Oppgave 2.2
# Hent en vektor for et ord og finn ut av hvilket ord det oftest
forekommer sammen med.
import numpy as np
word = "26"
vec = vectorizer.get_vector(word)

best_j = int(np.argmax(vec))
most_common_word = int_to_word[best_j]

print("Ord:", word)
print("Forekommer oftest med:", most_common_word)
print("Antall ganger:", vec[best_j])

Ord: 26
Forekommer oftest med: i
Antall ganger: 3545
```

Oppgave 2.3 – Vektordimensjonalitet (2 poeng)

Hva kan dere si om vektorene dere nå har laget? Er de høydimensjonale eller ikke? Nevn minst en fordel og en ulempe ved denne typen vektorer.

Oppgave 2.3 – Tekstbesvarelse

Svar: Vektorene vi har laget er høydimensjonale, siden hver vektor har én dimensjon per ord i vokabulæret (rundt 10 000 dimensjoner). Hver dimensjon representerer hvor ofte et annet ord forekommer i samme setning, noe som gir en direkte og tolkbar representasjon av kontekst. En fordel med denne typen tellevektorer er at de er enkle å forstå og implementere, og at de gir eksplisitt informasjon om hvilke ord som forekommer sammen. En ulempe er at de blir svært store og sparsomme, noe som gir høyt minnebruk og treg beregning. I tillegg fanger de kun opp overflate-samforekomster og ikke dypere semantiske relasjoner mellom ord.

Oppgave 2.4 – Måle distanser med cosinuslikhet (4 poeng)

Vi har nå laget en vektorrommodell av ordene i datasettet vårt, der hver kolonne representerer et ord i vokabularet vårt. Vi skal nå forsøke å tallfeste hvor like to vektorer, så vi kan se om dette stemmer overens med likheter mellom ord. Dette skal vi gjøre ved bruk av cosinuslikhet (engelsk: *cosine similarity*), der vi måler vinkelen mellom to vektorer.

Cosinuslikheten mellom to vektorer a og b er definert som:

$$cos(a,b)=\frac{a \cdot b}{\|a\| \|b\|}$$

Her er $a \cdot b$ prikkproduktet av de to vektorene, og $\|a\|$ og $\|b\|$ er normen (lengden) til henholdsvis vektor a og b.

Dere skal implementere funksjonen for cosinuslikhet på egenhånd. Det vil si at dere ikke skal importere ferdige funksjoner, men heller implementere dette direkte i Python. Dere kan derimot bruke pakken `math`, som har en rekke nyttige funksjoner for numeriske beregninger.

I denne oppgaven skal dere:

- Implementere funksjonen `cosine_similarity(a, b)` som tar inn to vektorer og returnerer cosinuslikheten.
- Bruke funksjonen til å finne cosinuslikheten mellom to ord dere selv velger. Skriv en kort kommentar på hva dere observerer.

Kommentarer og begrunnelser Oppgave 2.4

Bruker cosinus likhet for å måle likheten mellom to ordvektorer. Dette måler vinkelen mellom to vektorer, uten å bruke lengden. cosinuslikhet formelen er gitt ved:

$$cos(a,b)=\frac{a \cdot b}{\|a\| \|b\|}$$

Prikkproduktet er a*b og normen er sqrt(a^2). Dette blir implementert "for hånd" uten å bruke allerede eksisterende funksjoner. Det implementeres ved bruk av løkker og Math.sqrt() (square root). Fikk feilmelding på division by zero, så la inn en division by zero sjekk, som tar seg av dette for nullvektorer: if denominator = 0.

```
# Oppgave 2.4
import math
```

```python
def cosine_similarity(a, b):
    # Regn ut cosinuslikhet her
    dot = 0.0
    norm_a = 0.0
    norm_b = 0.0

    # Kalkulerer prikk produktet og normene
    for x, y in zip(a, b):
        x = float(x)
        y = float(y)
        dot += x * y
        norm_a += x * x
        norm_b += y * y

    denominator = math.sqrt(norm_a) * math.sqrt(norm_b)

    # Division by zero check
    if denominator == 0:
        return 0.0

    # Likhet
    return dot / denominator

# Eksempelbruk:
vec1 = vectorizer.get_vector("Norge")
vec2 = vectorizer.get_vector("Sverige")

likhet_res = cosine_similarity(vec1, vec2)
print(f"Cosinus likheten: {likhet_res:.3f}")
```

```
Cosinus likheten: 0.907
```

Oppgave 2.5 – Finne likheter mellom ord (5 poeng)

Vi skal nå bruke cosinuslikhet til å finne ord som er like hverandre i vektorrommet vårt. I cellen under finner dere to sett med ord i listene `WORDS` og `WORDS_SUBSET`. For hvert av ordene i `WORDS_SUBSET` skal dere finne de fem ordene fra `WORDS` som er likest, målt med cosinuslikhet. Lag en tabell i en markdown-celle der dere setter inn hvert ord i en kolonne og de fem likeste ordene i en annen kolonne. Dette kan dere gjøre på følgende måte:

```
| Ord    | 5 likeste |
| ------ | --------- |
| hund   | ...       |
| salat  | ...       |
| bil    | ...       |
...
```

Diskuter kort resultatene dere får. Virker det som at likere ord får høyere cosinuslikhet? Kom med noen eksempler og/eller moteksempler.

I denne oppgaven skal dere:

- Regne ut cosinuslikhet mellom hvert ord i `WORDS_SUBSET` og alle ordene i `WORDS`.
- Lage en tabell med de fem likeste ordene for hvert ord.
- Diskutere resultatene deres.

Kommentarer og begrunnelser Oppgave 2.5

I denne siste oppgaven, så brukes cosinuslikhet funksjonen til å finne de 5 mest like ordene for hvert element i WORDS_SUBSET listen, mot alle ordene i WORDS listen. Target-ord sin vektor blir sammelignet med kandidat-ord sin vektor. Parene blir lagerte på denne formen: (ord, likhet) i en ny liste, der de er sortert descending. Til slutt printes en emulert Markdown tabell slik som det blir bedt om i oppgaveteksten. Denne oppgaven gir en fin og oversiktlig måte å finne ut om vektorrommet fanger opp sematiske sammenhger, slik som om en katt er lik en hund, osv.

```
WORDS = [
    "gris", "elefant", "hund", "katt", "hest", "bjørn", "kylling",
"svin",
    "fisk", "salat", "brød", "ost", "smør", "melk", "bil", "moped",
"sykkel",
    "båt", "fly", "tog", "sykepleier", "lege", "ingeniør", "sjef",
    "sjåfør", "designer", "jente", "gutt", "mann", "kvinne", "Oslo",
"Trondheim",
    "Roma", "Egypt", "Mexico", "Galdhøpiggen", "Sørlandet", "Sahara",
"Bjørn",
    "Anne", "Jonas", "Erna", "Donald", "Barack", "Mohamed", "Angela"
]
WORDS_SUBSET = [
    "hund", "salat", "bil", "ingeniør", "sjef", "jente", "Oslo",
"Jonas"
]

# Oppgave 2.5
def top_5_similar(target_word, words, vectorizer):
    target_vec = vectorizer.get_vector(target_word)

    like = []
    for word in words:
        if word == target_word:
            continue
        word_vec = vectorizer.get_vector(word)
        lik = cosine_similarity(target_vec, word_vec)
        like.append((word, lik))

    # Sort on desc
    like.sort(key=lambda x: x[1], reverse=True)
    return like[:5]

resultater = {}
for words in WORDS_SUBSET:
```

```
    resultater[words] = top_5_similar(words, WORDS, vectorizer)


print("| Ord | 5 likeste (med cosinus likhet) |")
print("| --- | --- |")

for words in WORDS_SUBSET:
    top5 = resultater[words]
    pretty_pr = ", ".join([f"{word} ({lik:.3f})" for word, lik in
top5])
    print(f"| {words} | {pretty_pr} |")
```

```
| Ord | 5 likeste (med cosinus likhet) |
| --- | --- |
| hund | hest (0.978), båt (0.977), katt (0.976), bil (0.973), gris
(0.971) |
| salat | ost (0.954), brød (0.947), kylling (0.947), smør (0.947),
melk (0.901) |
| bil | båt (0.982), hund (0.973), sjåfør (0.971), mann (0.965), hest
(0.962) |
| ingeniør | designer (0.968), lege (0.964), sykepleier (0.957), hest
(0.955), gris (0.955) |
| sjef | ingeniør (0.939), lege (0.939), fisk (0.936), Anne (0.930),
designer (0.929) |
| jente | gutt (0.970), kvinne (0.963), mann (0.960), bil (0.916),
sjåfør (0.913) |
| Oslo | Trondheim (0.990), Mexico (0.968), Egypt (0.967), Roma
(0.953), Anne (0.944) |
| Jonas | Anne (0.777), Bjørn (0.775), Egypt (0.767), lege (0.767),
fisk (0.764) |
```

Oppgave 2.5 – Tekstbesvarelse

Legg inn tabellen din her. Hva ser du av resultatene?

Svar: Tabellen jeg får: | Ord | 5 likeste (med cosinus) | | --- | --- | | hund | hest (0.978), båt (0.977), katt (0.976), bil (0.973), gris (0.971) | | salat | ost (0.954), brød (0.947), kylling (0.947), smør (0.947), melk (0.901) | | bil | båt (0.982), hund (0.973), sjåfør (0.971), mann (0.965), hest (0.962) | | ingeniør | designer (0.968), lege (0.964), sykepleier (0.957), hest (0.955), gris (0.955) | | sjef | ingeniør (0.939), lege (0.939), fisk (0.936), Anne (0.930), designer (0.929) | | jente | gutt (0.970), kvinne (0.963), mann (0.960), bil (0.916), sjåfør (0.913) | | Oslo | Trondheim (0.990), Mexico (0.968), Egypt (0.967), Roma (0.953), Anne (0.944) | | Jonas | Anne (0.777), Bjørn (0.775), Egypt (0.767), lege (0.767), fisk (0.764) |

Resultatene viser at ord som er semantisk relaterte ofte får høy cosinuslikhet. For eksempel er hund mest lik hest, katt og gri, som alle er dyr, og salat er mest lik ost, brød og smør som alle er matvarer. Tilsvarende ser vi at jente er mest lik gutt, kvinne og mann, som er forventet. Stedsnavn som Oslo er mest lik andre stedsnavn som Trondheim og Roma. Samtidig finnes det enkelte mindre intuitive sammenhenger, som at infeniør er relativt lik hest og gris, noe som jeg går ut ifra er tilfeldige sammenhenger som har forekommet i datasettet.

For å konkludere så er resultatene for cosinuslikhet i de tellerbaserte vektorene representasjoner for meningsfulle relasjoner i mange tilfeller, men de kan også inneholde uønsket og uforutsett støy.

NB!

Husk å kjøre gjennom hele notebooken fra toppen av før dere leverer obligen!